

Дерево устройств

Лекция и практическое занятие

Подольская Н.А.

Конфигурация платформы

Платформы могут использоваться для разных целей и иметь самую разную конфигурацию.

На PC ОС получает информацию о конфигурации платформы от BIOS, которая разрабатывается специально для каждой платформы. Сама же BIOS при загрузке определяет конфигурацию по содержимому регистров MSR (Machine Specific Registers), адреса которых hardcoded, и содержимого PCI псевдо-устройств.

Для Линукса, работающего на SoC (система на чипе) было решено держать информацию о конфигурации платформы по фиксированному адресу оперативной памяти, куда ее записывает загрузчик перед тем, как передать управление ядру.

Доступ к конфигурации платформы в Линуксе

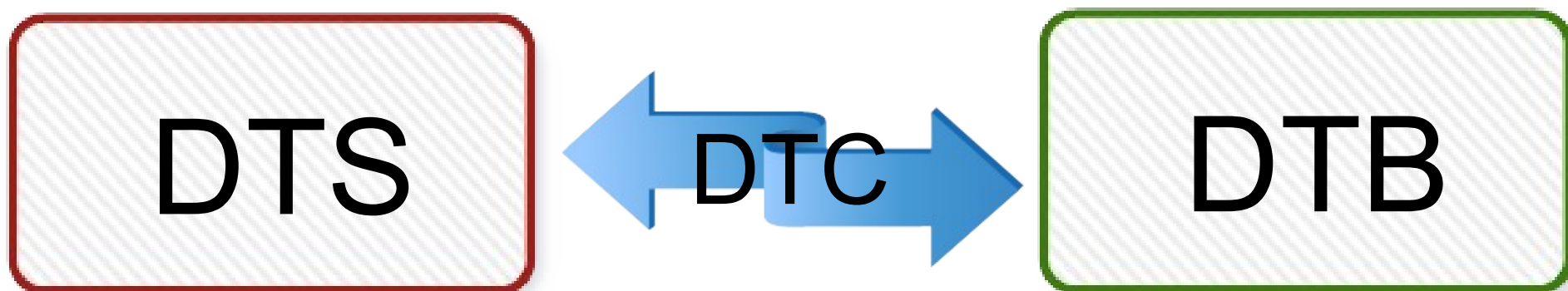
Итак, информация о дереве устройств находится в оперативной памяти, по фиксированному адресу.

Каждая программа ядра может извлекать любую информацию из любой части дерева устройств.

Программист может размещать те параметры устройств, которые ему нужны, в тех местах дерева, которые сочтет подходящими для этого.

Форматы дерева устройств

- 1) Исходный текст DTS: Device Tree Source
- 2) Двоичный формат DTB: Device Tree Blob



Компилятор DTC используется для получения DTB из DTS и наоборот:

```
$ dtc -I dts -O dtb -o out.dtb in.dts
```

```
$ dtc -I dtb -O dts -o out.dts out.dts
```

Расположение дерева устройств

DTS и DTB файлы для различных платформ находятся в папке `arch/<cpu>/boot/dts`.

DTS-файл `am335x-boneblack.dts` для BlackBone Black:

```
/dts-v1/;
#include "am33xx.dtsi"
#include "am335x-bone-common.dtsi"
&ldo3_reg {
    regulator-min-microvolt = <1800000>;
    ...
}
...
```

Структура дерева устройств

```
/dts-v1/;  
/  
    compatible = "ti,am33xx";  
    interrupt-parent = <&intc>;  
    aliases {  
        ...  
        serial0 = &uart0;  
        serial1 = &uart1;  
        ...  
    };  
    cpus {  
        [ ... CPU definitions ... ]  
    };  
    [ ... Peripheral definitions ... ]
```

Каждый узел (node) дерева соответствуют устройству. Внутри узлов могут находиться другие узлы. Корневой узел дерева устройств помечен знаком '/'.

Формат DTS

```
/dts-v1/;
/ {
    node1 {
        a-string-property = "A string";
        a-string-list-property = "first string", "second string";
        a-byte-data-property = [0x01 0x23 0x34 0x56];
        child-node1 {
            first-child-property;
            second-child-property = <1>;
            a-string-property = "Hello, world";
        };
        child-node2 {
        };
    };
    node2 {
        an-empty-property;
        a-cell-property = <1 2 3 4>; /* each number (cell) is a uint32 */
        child-node1 {
        };
    };
};
```

Описание дерева устройств гипотетической машины

Конфигурация гипотетической машины

- Один 32-битный процессор ARM
- Шина процессора, подсоединенная к последовательному порту, отображенному на память, контроллер шины SPI, контроллер I2C, контроллер прерываний и мост внешней шины
- 256MB RAM, расположенный по адресу 0
- 2 последовательных порта с адресами 0x101F1000 и 0x101F2000
- Контроллер GPIO, адрес 0x101F3000
- Контроллер SPI, адрес 0x10170000
- Слот MMC в пином SS, присоединенный к GPIO #1
- Мост внешней шины со следующими устройствами:
 - SMC SMC9111 Ethernet, подсоединенный к внешней шине, с адресом 0x10100000
 - Контроллер I2C, с адресом 0x10160000, с устройствами:
 - Часы реального времени Maxim DS1338. Отвечают slave адресу 1101000 (0x58)
 - 64MB NOR flash, адрес 0x30000000

Минимальное дерево

```
/dts-v1/;  
  
/{  
    compatible = "acme,coyotes-revenge";  
};
```

Имя системы (compatible) – строка формата
<производитель>,<модель>

Описание процессоров

```
/dts-v1/;
/ {
    compatible = "acme,coyotes-revenge";

    cpus {
        cpu@0 {
            compatible = "arm,cortex-a9";
        };
        cpu@1 {
            compatible = "arm,cortex-a9";
        };
    };
};
...
```

Процессор: двухъядерный ARM Cortex A9

Описание периферии

...

```
serial@101F0000 {  
    compatible = "arm,pl011";  
};  
serial@101F2000 {  
    compatible = "arm,pl011";  
};  
gpio@101F3000 {  
    compatible = "arm,pl061";  
};  
interrupt-controller@10140000 {  
    compatible = "arm,pl190";  
};  
spi@10115000 {  
    compatible = "arm,pl022";  
};
```

```
external-bus {  
    ethernet@0,0 {  
        compatible = "smc,smc91c111";  
    };  
    i2c@1,0 {  
        compatible = "acme,a1234-i2c-  
bus";  
        rtc@58 {  
            compatible = "maxim,ds1338";  
        };  
    };  
    flash@2,0 {  
        compatible =  
"samsung,k8f1315ebm", "cfi-flash";  
    };  
};
```

Адресация

Для описания адресов устройств используются опции:

- reg
- #address-cells
- #size-cells

Каждое адресуемое устройство получает поле reg, описывающее пространство адресов используемых устройством. Это набор кортежей (tuples), содержащих начальные адреса и размеры используемых сегментов адресов

В “родительском” узле задаются параметры кортежей: сколько они содержат 32-битных чисел для адреса и для размера

Замечание. Если узел содержит поле reg, то в его имя должно входить значение этого поля (см.пример на след.слайде).

Адресация процессора

```
cpus {  
    #address-cells = <1>;  
    #size-cells = <0>;  
    cpu@0 {  
        compatible = "arm,cortex-a9";  
        reg = <0>;  
    };  
    cpu@1 {  
        compatible = "arm,cortex-a9";  
        reg = <1>;  
    };  
};
```

Устройства, отображенные на память

```
/dts-v1/;

/ {
    #address-cells = <1>;
    #size-cells = <1>;
    ...
    serial@101f0000 {
        compatible = "arm,pl011";
        reg = <0x101f0000 0x1000 >;
    };

    serial@101f2000 {
        compatible = "arm,pl011";
        reg = <0x101f2000 0x1000 >;
    };
};
```

```
gpio@101f3000 {
    compatible = "arm,pl061";
    reg = <0x101f3000 0x1000
        0x101f4000 0x0010>;
};

interrupt-controller@10140000 {
    compatible = "arm,pl190";
    reg = <0x10140000 0x1000 >;
};

spi@10115000 {
    compatible = "arm,pl022";
    reg = <0x10115000 0x1000 >;
};
...
};
```

Внешняя шина

```
external-bus {  
    #address-cells = <2>  
    #size-cells = <1>;  
    ethernet@0,0 {  
        compatible = "smc,smc91c111";  
        reg = <0 0 0x1000>;  
    };  
    i2c@1,0 {  
        compatible = "acme,a1234-i2c-bus";  
        reg = <1 0 0x1000>;  
        rtc@58 {  
            compatible = "maxim,ds1338";  
        };  
    };  
    flash@2,0 {  
        compatible = "samsung,k8f1315ebm", "cfi-flash";  
        reg = <2 0 0x40000000>;  
    };  
};
```

Устройства
на внешней
шине имеют
два адреса
(первый для
выбора
чипа)

Устройства, не отображенные на память

Устройства, не отображенные на память, тоже могут иметь поле reg, но его содержимое не используется процессором напрямую:

```
i2c@1,0 {  
    compatible = "acme,a1234-i2c-bus";  
    #address-cells = <1>;  
    #size-cells = <0>;  
    reg = <1 0 0x1000>;  
    rtc@58 {  
        compatible = "maxim,ds1338";  
        reg = <58>;  
    };  
};
```

Запись и чтение таких устройств производится через вызовы функции драйвера. Соответствующий драйвер должен быть ассоциирован с шиной I2C по значению свойства compatible (acme,a1234-i2c-bus)

Трансляция адресов

```
external-bus {
    #address-cells = <2>
    #size-cells = <1>;
    ranges = <0 0 0x10100000 0x10000 // Chipselect 1, Ethernet
              1 0 0x10160000 0x10000 // Chipselect 2, i2c controller
              2 0 0x30000000 0x1000000>; // Chipselect 3, NOR Flash

    ethernet@0,0 {
        compatible = "smc,smc91c111";
        reg = <0 0 0x1000>;
    };

    i2c@1,0 {
        compatible = "acme,a1234-i2c-bus";
        #address-cells = <1>;
        #size-cells = <0>;
        reg = <1 0 0x1000>;
        rtc@58 {
            compatible = "maxim,ds1338";
            reg = <58>;
        };
    };
};
```

Описание прерываний

Для описания прерываний используются следующие поля:

- `interrupt-controller` – пустое поле, обозначающее, что устройство получает прерывания
- `#interrupt-cells` – поле контроллера прерываний, определяет количество ячеек в описании прерываний
- `interrupt-parent` – имя узла контроллера прерываний, на номера которого ссылаются другие вложенные узлы
- `interrupts` – номер и тип прерывания

Пример описания прерываний (1/2)

```
/dts-v1/;
/ {
    compatible = "acme,coyotes-
revenge";
    #address-cells = <1>;
    #size-cells = <1>;
    interrupt-parent = <&intc>;
    cpus { ...
    };
    serial@101f0000 {
        compatible = "arm,pl011";
        reg = <0x101f0000 0x1000 >;
        interrupts = < 1 0 >;
    };
    serial@101f2000 {
        compatible = "arm,pl011";
        reg = <0x101f2000 0x1000 >;
        interrupts = < 2 0 >;
    };
};
```

```
gpio@101f3000 {
    compatible = "arm,pl061";
    reg = <0x101f3000 0x1000
        0x101f4000 0x0010>;
    interrupts = < 3 0 >;
};
intc: interrupt-
controller@10140000 {
    compatible = "arm,pl190";
    reg = <0x10140000 0x1000 >;
    interrupt-controller;
    #interrupt-cells = <2>;
};
spi@10115000 {
    compatible = "arm,pl022";
    reg = <0x10115000 0x1000 >;
    interrupts = < 4 0 >;
};
...
```

Пример описания прерываний (2/2)

```
external-bus {
    #address-cells = <2>
    #size-cells = <1>;
    ranges = ...
    ethernet@0,0 {
        compatible =
"smc,smc91c111";
        reg = <0 0 0x1000>;
        interrupts = < 5 2 >;
    };
};
```

```
    i2c@1,0 {
        compatible =
"acme,a1234-i2c-bus";
        #address-cells = <1>;
        #size-cells = <0>;
        reg = <1 0 0x1000>;
        interrupts = < 6 2 >;
        rtc@58 {
            compatible =
"maxim,ds1338";
            reg = <58>;
            interrupts = < 7 3 >;
        };
    };
};
...
};
};
```

Использование DTS в загрузчике

```
chosen {  
    bootargs = "root=/dev/nfs rw  
nfsroot=192.168.1.1 console=ttyS0,115200";  
};
```

Узел chosen может использоваться для передачи данных между прошивкой и ОС. Например, для передачи параметров загрузки.

Часто загрузчик (U-boot) сам заполняет это поле в DTB.

Команда bootm загрузчика U-boot используется для задания адресов образа ядра, образа ramdisk и образа DTB:

```
bootm <Linux image addr> <ramdisk addr> <dtb address>
```

Связь DTS и драйверов ядра (1/3)

В коде драйвера задается имя, соответствующее полю compatible из DTS:

```
static const struct of_device_id fs_enet_match[] __devinitdata = {
    {
        .compatible = "fsl,cpm1-scc-enet",
        .data = (void *)&fs_scc_ops,
    },
    {
        .compatible = "fsl,cpm2-scc-enet",
        .data = (void *)&fs_scc_ops,
    },
}
};

MODULE_DEVICE_TABLE(of, fs_enet_match);
```

Связь DTS и драйверов ядра (2/3)

Драйвер регистрирует себя как OF (Open Firmware) platform driver:

```
static struct platform_driver fs_enet_driver = {  
    .driver = {  
        .name = "fs_enet",  
        .of_match_table = fs_enet_match,  
    },  
    .probe = fs_enet_probe,  
    .remove = fs_enet_remove,  
};  
module_platform_driver(fs_enet_driver);
```


Связь DTS и драйверов ядра (3/3)

Функция probe получает указатель структуры ofdev, указывающий на данные из DTS. Оттуда извлекаются все поля DTS:

```
static int fs_enet_probe(struct platform_device *ofdev) {  
    ...  
    data = of_get_property(ofdev->dev.of_node, "fsl,cpm-  
command", &len);  
    fpi->cp_command = *data;  
    ...  
}
```

Файл arch/powerpc/boot/dts/mpc885ads.dts:

```
ethernet@a40 {  
    device_type = "network";  
    compatible = "fsl,mpc885-scc-enet","fsl,cpm1-scc-enet";  
    ...  
    fsl,cpm-command = <0x80>;  
};
```

Практическое занятие

Список устройств в Линуксе

/dev - каталог, содержащий псевдо-файлы устройств. В Linux все физические устройства, как главные, так и периферийные, представляют собой файлы особого типа, в которые система может записывать данные и из которых она может их считывать. Работать с этими файлами нельзя, поскольку запись неправильных данных в файл устройства может повредить устройство или хранящиеся на нём данные

На BeagleBone видим только один последовательный порт:

```
$ ls /dev
```

```
...
```

```
ttyO0
```

```
ttyS0
```

```
...
```

```
$
```

Описание последовательного порта в BeagleBone

Файл am335x-bone-common.dtsi:

```
/ {  
    Model = "TI AM335x BeagleBone"  
    ...  
};  
  
&uart0 {  
    pinctrl-names = "default";  
    pinctrl-0 = <&uart0_pins>;  
    status = "okay";  
};  
  
&usb {  
    status = "okay";  
    ...  
};
```

Упражнение

Задание. Добавить порт ttyO1 в список устройств /dev

Состояние списка устройств

После загрузки ОС:

```
root@am335x-evm:~# ls /dev/ttyO*
```

```
/dev/ttyO0
```

```
root@am335x-evm:~#
```

Добавляем устройство в DTSI файл

Файл `./ti-processor-sdk-linux-am335x-evm-01.00.00.00/board-support/linux-3.14.26-g2489c02/arch/arm/boot/dts/am335x-bone-common.dtsi`:

```
&uart0 {  
    pinctrl-names = "default";  
    pinctrl-0 = <&uart0_pins>;  
    status = "okay";  
};
```

```
&uart1 {  
    status = "okay";  
};
```

Компилируем правки

```
./ti-processor-sdk-linux-am335x-evm-  
01.00.00.00$ make
```


Записываем базу деревьев в образ ОС

```
./ti-processor-sdk-linux-am335x-evm-01.00.00.00/  
board-support/linux-3.14.26-g2489c02/arch/arm/  
boot/dts$ sudo rm /media/rootfs/boot/am335x-  
boneblack.dtb
```

```
./ti-processor-sdk-linux-am335x-evm-01.00.00.00/  
board-support/linux-3.14.26-g2489c02/arch/arm/  
boot/dts$ sudo cp am335x-boneblack.dtb  
/media/rootfs/boot/
```

Проверяем результат

После загрузки ОС:

```
root@am335x-evm:~# ls /dev/ttyO*
```

```
/dev/ttyO0 /dev/ttyO1
```

```
root@am335x-evm:~#
```

BackUp

Загрузка beagle bone - uboot

Запуск ядра из памяти осуществляет команда:

bootz \${loadaddr} - \${fdtaddr}

*Узнаем адреса в памяти где должны
находиться образы DTB и ядра. В
командной строке Uboot:*

U-Boot# printenv fdtaddr

fdtaddr=0x88000000

U-Boot# pri loadaddr

loadaddr=0x82000000

Запуск ядра с DTS

ATTENTION: Для того, чтобы ядро после запуска нашло файловую систему, нужно, чтобы в переменной bootarg в Uboot была задана правильная командная строка ядру.

- В нашем случае (плата BeagleBone) запуск ядра с DTS производится так:*

- bootz 0x82000000 - 0x88000000*

Запуск собственного ядра со своим DTV

- Для того, чтобы разместить собранное нами ядро и DTV файл от него в памяти системы, можно воспользоваться загрузкой их по сети.
- Для этого используется TFTP (trivial file transfer protocol). На хосте должен быть запущен tftp сервер и образы `zulmage.beaglebone` `beaglebone.dtb` следует разместить в его директории для ресурсов `/tftpboot`
- Бутлоадеру Uboot при помощи задания переменных следует сообщить IP адрес хоста для закачки по tftp и задать собственный IP адрес таргета:
 - `setenv serverip 192.168.1.xxx`
`setenv ipaddr 192.168.1.yyy`

Загрузка по tftp

- Загрузим собранные нами образы ядра и DTB, по тем же адресам, по которым они загружаются со встроенной флэш памяти.
- *tftp zulmage.beaglebone 0x82000000*
- *tftp beaglebone.dtb 0x88000000*

Запуск ядра

bootz 0x82000000 – 0x88000000